Ministerul Educaţiei şi Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică şi Microelectronică

Departamentul *Ingineria Software*

Report on laboratory work no. 4

Course: Operating Systems

Topic: The xv6 Shell from the Outside

Done by: st. gr. FAF-241, Lungu Ilie

Checked by: *Relitman Patricia*

Chişinău – 2026

Contents

Purpose of the work **2**

1 PART 1 – FIX SLEEP FOR THE CURRENT XV6 **2**
1.1 Which name does the tree use? 2

2 PART 2 – BASH VERSUS THE XV6 SHELL **3**
2.1 ls after cd d 5
2.2 Observe 5

3 PART 3 – WHO EXPANDS *? A PROGRAM THAT PRINTS ITS ARGV **6**
3.1 Observe 7

4 PART 4 – READ THE SHELL **7**
4.1 Observe 8

Conclusions **9**

PURPOSE OF THE WORK

The purpose of this laboratory work is to see what a shell really has to do, by using the shell of xv6 side by side with `bash` and noting the differences. The whole user interface of xv6 is one program, `user/sh.c`, of 499 lines, so every feature that `bash` has and xv6 lacks becomes visible. First, the program `sleep` from Lab 2 is adapted to the current xv6 (the system call `sleep()` was renamed `pause()`). Then the same command lines are run in both shells, a small program that prints its arguments shows who expands `*` and who removes the quotes, and finally the relevant parts of `sh.c` are read. The experiments were run on the current xv6-riscv tree (commit `06aad25`) in QEMU, and on Linux with `bash`; the outputs are reproduced as figures, and the answers to all the *Observe* boxes are given in the corresponding parts.

1 PART 1 – FIX SLEEP FOR THE CURRENT XV6

1.1 Which name does the tree use?

In August 2025 the authors of xv6 renamed the system call `sleep()` to `pause()`. The declaration in `user/user.h` shows which name the current tree uses:

```
$ cd ~/xv6-riscv
$ grep -n "pause\|sleep" user/user.h
25:int pause(int);
```

Figure 1.1 – The grep result in user.h

Only `pause` is declared (line 25) and there is no `sleep`, so a `sleep.c` written for the old tree that calls `sleep()` would not build any more: the compiler would report an undeclared function. The call was therefore changed to `pause()`; the file is shown in Figure 1.2.

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if(argc != 2){
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }
  pause(atoi(argv[1])); // was sleep() before Aug 2025
  exit(0);
}
```

Figure 1.2 – The updated sleep.c

The program checks that exactly one argument was given; otherwise it prints the usage message on the standard error (file descriptor 2) and exits with the code 1. The argument is converted with `atoi` and passed to `pause`, which suspends the process for that number of ticks (a tick is one timer interrupt, about a tenth of a second here).

For the program to be placed in the file system image, the line `$U/_sleep\` was added to `UPROGS` in the `Makefile` (Figure 1.4). Each line starts with a TAB and ends with a backslash, with nothing after the backslash.

```
$ make qemu
...
init: starting sh
$ sleep 20
$ sleep
usage: sleep <ticks>
```

Figure 1.3 – sleep running in xv6, with and without an argument

`sleep 20` waits about two seconds and returns to the prompt without printing anything, and `sleep` without an argument prints the usage message. The program builds and works on the current tree. QEMU was left with Ctrl-A, then X.

```
UPROGS=\
    $U/_cat\
    $U/_echo\
...
    $U/_dorphan\
    $U/_sync\
    $U/_sleep\
    $U/_args\
```

Figure 1.4 – The UPROGS lines for sleep and args in the Makefile

2 PART 2 – BASH VERSUS THE XV6 SHELL

Every command of the table was run twice: in `bash` on Linux, in an empty test folder that contains only a file named `README` (`mkdir ~/shtest && cd ~/shtest && touch README`), and at the `$` prompt of xv6 (`make qemu`). The results are shown in Figure 2.1 (bash) and Figure 2.2 (xv6) and are summed up in Table 2.1.

```
$ echo hello > f ; cat < f
hello
$ ls | grep READ
README
$ (echo a; echo b) | wc
      2       2       4
$ echo one ; echo two
one
two
$ echo bg &
[1] 1590
$ bg
$ ls *
README  f
$ echo $HOME
/root
$ echo "a b"
a b
$ mkdir d ; cd d ; ls ..
README  d  f
$ cd ..
$ echo hello
hello
$ <Up arrow>
echo hello
hello
```

Figure 2.1 – The commands run in bash

```
$ echo hello > f ; cat < f
hello
$ ls | grep READ
README          2 2 2441
$ (echo a; echo b) | wc
2 2 4
$ echo one ; echo two
one
two
$ echo bg &
$ bg
$ ls *
ls: cannot open *
$ echo $HOME
$HOME
$ echo "a b"
"a b"
$ mkdir d ; cd d ; ls ..
```

```
exec cd failed
.              1 1 1024
..             1 1 1024
README         2 2 2441
cat            2 3 35376
...
$ echo hello
hello
$ <Up arrow>
exec ^[[A failed
```

Figure 2.2 – The same commands run in xv6

The listing printed by `ls ..` in the last command of Figure 2.2 is shortened (the dots stand for the other programs of the root directory). The `ls` of xv6 prints, for every entry, the name, the type (1 = directory, 2 = file, 3 = device), the inode number and the size; this is why `ls | grep READ` prints a whole line, not only the name.

Table 2.1 – The commands of Part 2 in bash and in xv6, and why.

Command	**bash**	**xv6**	**Why**
`echo hello > f ; cat < f`	`hello`	`hello`	Redirection `>` and `<`, and the list `;`: both shells have them.
`ls | grep READ`	`README`	`README 2 2 2441`	Pipe `|`: both. The line differs because the `ls` of xv6 prints type, inode and size after the name.
`(echo a; echo b) | wc`	`2 2 4`	`2 2 4`	Grouping with parentheses: both. `wc` receives the output of both `echo` commands (2 lines, 2 words, 4 bytes).
`echo one ; echo two`	`one` / `two`	`one` / `two`	Sequence `;`: both.
`echo bg &`	`[1] 1590`, then `bg`	`bg` (after the prompt)	Background `&`: both start the command without waiting. Only bash has job control (job number, `jobs`, `kill %1`); in xv6 the shell just does not call `wait`.
`ls *`	`README f`	`ls: cannot open *`	Globbing: bash expands `*` to the names; xv6 passes the character `*` to `ls`, which looks for a file named `*`.
`echo $HOME`	`/root`	`$HOME`	Variables: bash replaces `$HOME`; xv6 has no variables and prints the text.
`echo "a b"`	`a b`	`"a b"`	Quotes: bash removes them; xv6 does not know them and keeps them as ordinary characters.
`mkdir d ; cd d ; ls ..`	`README d f`	`exec cd failed`, then the listing of `/`	PATH and built-in `cd`: in bash `cd` is a built-in and works inside a list. In xv6 `cd` is recognized only at the start of a whole line, so here a child process tries to run a program called `cd` and fails; `ls ..` then runs in the old directory (see Part 4).

Command	bash	xv6	Why
arrow up, Tab, Ctrl+R	work (arrow up repeated `echo hello`)	arrow up: `exec ^[[A failed`	History and completion: bash reads the line with a line-editing library. xv6 `sh` reads a whole line from the console, which only knows Backspace and Ctrl-U; the arrow key arrives as the characters `ESC [ A` and is taken as a program name.

Tab and Ctrl+R were not typed in the automated bash run; they work in bash as in Lab 3 (Part 1), and in xv6 there is no code at all for them. The last row of the table uses the arrow key, which was tested in both shells.

2.1 ls after cd d

```
$ mkdir d
$ echo hi > d/x
$ cd d
$ ls
exec ls failed
$ /ls
.              1 26 48
..             1 1 1024
x              2 27 3
$ /ls ..
.              1 1 1024
..             1 1 1024
README         2 2 2441
cat            2 3 35376
...
d              1 26 48
$ cd ..
```

Figure 2.3 – ls failing after cd d, and the working alternative

2.2 Observe

1. **Common features and features only of bash.** Both shells have: running a program with arguments, input and output redirection (`<`, `>`, and `»` in xv6), pipes (`|`), command lists (`;`), grouping with parentheses and background execution (`&`); `cd` is a built-in in both. Only bash has: globbing (`*`, `?`, `[...]`), variables (`$HOME`) and the expansions `$(...)` and `$((...))`, quoting (single and double quotes, backslash), `&&` and `||`, a `PATH` to find the programs, job control (`jobs`, `fg`, `kill %1`), command history, line editing and Tab completion, scripts with `if`, `for` and functions, and many more built-ins.

2. **Why does `ls` stop working after `cd d`?** The xv6 shell has no `PATH`. It calls `exec("ls", ...)` with the name exactly as typed, and `exec` looks for a file called `ls` in the *current* directory. The programs of xv6 live in the root directory `/` (the listing of `/` in Figure 2.3 shows `ls`, `cat`, `echo`, ...), so `ls` works only while the current directory is `/`. After `cd d` there is no file `ls` in `d`, and the shell prints `exec ls failed`. The fix is to give the full path of the program: `/ls` (or `/ls ..`) runs it from inside `d`, as Figure 2.3 shows. The same holds for every other program: in `d`, even `/ls | grep x` fails with `exec grep failed`, because `grep` is also looked for in `d`; it has to be written `/grep`.

3. **Which missing feature would I miss most, and which is the easiest to add?** In daily work I would miss most the command history with line editing and Tab completion: long commands and file names would have to be typed again and again (globbing and variables would come second). The easiest to add is a minimal `PATH`: when `exec(argv[0], ...)` fails, the shell can build the name `"/" + argv[0]` and call `exec` again, which is only a few lines in the `EXEC` case of `runcmd` and would solve the problem of `ls` after `cd d`. The hardest ones are history and completion, because the shell would have to read the keyboard character by character instead of whole lines.

3 PART 3 – WHO EXPANDS *? A PROGRAM THAT PRINTS ITS ARGV

The program `args` prints the number of arguments it received and every argument between square brackets, so that the spaces and the special characters can be seen exactly. For xv6 it is `user/args.c`, added to `UPROGS`; for Linux the same program was written as `args_linux.c`, with `<stdio.h>` instead of the two xv6 headers and `return 0` instead of `exit(0)`, and compiled with `gcc -o args args_linux.c`.

```
// user/args.c
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  printf("argc = %d\n", argc);
  for(int i = 0; i < argc; i++)
    printf("argv[%d] = [%s]\n", i, argv[i]);
  exit(0);
}

// Makefile, in UPROGS:
    $U/_args\
```

Figure 3.1 – args.c and the UPROGS line

```
$ args one   two three
argc = 4
argv[0] = [args]
argv[1] = [one]
argv[2] = [two]
argv[3] = [three]
$ args *.c "x y"
argc = 4
argv[0] = [args]
argv[1] = [*.c]
argv[2] = ["x]
argv[3] = [y"]
$ args $HOME
argc = 2
argv[0] = [args]
argv[1] = [$HOME]
$ args 'single quoted'
argc = 3
argv[0] = [args]
argv[1] = ['single]
argv[2] = [quoted']
```

Figure 3.2 – args output in xv6

```
$ gcc -o args args_linux.c
$ ls *.c
a.c  args_linux.c  b.c
```

```
$ ./args one   two three
argc = 4
argv[0] = [./args]
argv[1] = [one]
argv[2] = [two]
argv[3] = [three]
$ ./args *.c "x y"
argc = 5
argv[0] = [./args]
argv[1] = [a.c]
argv[2] = [args_linux.c]
argv[3] = [b.c]
argv[4] = [x y]
$ ./args $HOME
argc = 2
argv[0] = [./args]
argv[1] = [/root]
$ ./args 'single quoted'
argc = 2
argv[0] = [./args]
argv[1] = [single quoted]
```

Figure 3.3 – args_linux output in bash

In bash the test folder contained the three files `a.c`, `args_linux.c` and `b.c`, so the number of arguments of the second line depends on the folder. In xv6 the line `args *` (not shown in the figure) gives `argc = 2` and `argv[1] = [*]`.

3.1 Observe

1. **The outputs of the second line.** In xv6 the program received `argc = 4`: `args`, `*.c`, `"x` and `y"` (3 arguments after the program name). In bash it received `argc = 5`: `./args`, `a.c`, `args_linux.c`, `b.c` and `x y` (4 arguments after the program name; the number of file names depends on the folder).

2. **Who turns `*.c` into file names and removes the quotes?** The *shell*, not the program and not the kernel. The program prints exactly what is in its `argv`. The kernel does not interpret the arguments either: `exec` only copies the array of strings from the shell to the new program. In bash the shell replaced `*.c` with the three names and removed the quotes, so that `x y` became one argument. The xv6 shell did neither: it passed `*.c` unchanged, and since it does not know quotes, the characters `"` stayed in the words `"x` and `y"`, and the text was split at the space. Likewise bash replaced `$HOME` with `/root`, while xv6 passed the text `$HOME`.

3. **What happened to the extra spaces in `one two three`?** They disappeared in both shells: the program received the three words `one`, `two`, `three` and `argc = 4`. The spaces are only separators between words; the shell splits the line at any run of blanks and does not pass them on, so the program cannot know how many spaces were typed. To keep spaces inside an argument, bash needs quotes (`"x y"`), and the xv6 shell has no way to do it.

4 PART 4 – READ THE SHELL

The commands of the assignment were run inside `~/xv6-riscv` and the parts of `user/sh.c` they point to were read.

```
$ wc -l user/sh.c
499 user/sh.c
$ grep -n "#define" user/sh.c
8:#define EXEC  1
```

```
9:#define REDIR 2
10:#define PIPE  3
11:#define LIST  4
12:#define BACK  5
14:#define MAXARGS 10
```

Figure 4.1 – The command kinds defined in sh.c

```
$ grep -n "case " user/sh.c
75:  case EXEC:
83:  case REDIR:
93:  case LIST:
101:  case PIPE:
125:  case BACK:
284:  case 0:
286:  case '|':
287:  case '(':
288:  case ')':
289:  case ';':
290:  case '&':
291:  case '<':
294:  case '>':
391:    case '<':
394:    case '>':
397:    case '+': // >>
469:  case EXEC:
475:  case REDIR:
481:  case PIPE:
487:  case LIST:
493:  case BACK:
```

Figure 4.2 – Where each kind is run

Lines 75–125 belong to the function `runcmd`, which executes a command tree: this is where each kind is run. Lines 284–294 are in the tokenizer `gettoken`, where the special characters are recognized, lines 391–397 in the parsing of redirections, and lines 469–493 in `nulterminate`.

```
$ grep -n "cd" user/sh.c | head
170:        fprintf(2, "cannot cd %s\n", cmd + 3);
$ sed -n 166,171p user/sh.c
    if (cmd[0] == 'c' && cmd[1] == 'd' && cmd[2] == ' ') {
      // Chdir must be called by the parent, not the child.
      cmd[strlen(cmd) - 1] = 0; // chop \n
      if (chdir(cmd + 3) < 0)
        fprintf(2, "cannot cd %s\n", cmd + 3);
    } else {
```

Figure 4.3 – The code that handles cd

```
$ grep -n "'\*'\|'\$'\|glob\|getenv\|environ" user/sh.c
$ echo $?
1
```

Figure 4.4 – No code for * or $ in sh.c

4.1 Observe

1. **The five kinds of command.** (1) `EXEC` – a simple command, a program name with its arguments (no special character; the words are separated by blanks); (2) `REDIR` – redirection of a command, produced by `<`, `>` and `»`; (3) `PIPE` – two commands joined by `|`; (4) `LIST` – two commands one after another, produced by `;`; (5) `BACK` – a command run in the background, produced by `&`. The parentheses `(` and `)` do not make a kind of their own: they only group a command list (as in `(echo a; echo b) | wc`).

2. **Where is `cd` handled, and why is it not a separate program?** In the function `main`, in the loop that reads the commands (lines 166–171 of `sh.c`): if the line starts

with `cd` , the shell calls `chdir` itself, in the parent process; every other line is run by a child created with `fork`. The comment in the code says it: `chdir` must be called by the parent. The current directory belongs to a process; if `cd` were a program, it would change the directory of its own child process, which ends at once, and the shell would stay where it was. This also explains `exec cd failed` in Part 2: the test is made only on the whole line, so in `mkdir d ; cd d ; ls ..` the part `cd d` is run by a child through `runcmd`, which looks for a program called `cd` and does not find it.

3. **Code for `*` or `$`.** None: the search of Figure 4.4 finds nothing. The tokenizer `gettoken` knows only the blanks and the symbols `<|>&;()`; any other characters, including `*`, `$` and the quotes, are ordinary characters of a word. This explains Part 2: `ls *` asks for a file called `*`, `echo $HOME` prints the text `$HOME`, and `echo "a b"` prints the quotes. It also explains Part 3, where the arguments reach the program exactly as typed, and it confirms that in bash all this is done by the shell.

CONCLUSIONS

In this laboratory work the shell of xv6 was studied from the outside and compared with `bash`. The program `sleep` was adapted to the current xv6, where the system call is called `pause()`, and it works with and without an argument. The experiments showed that the xv6 shell is very small (499 lines) but has the essential mechanisms of a shell: running programs with `fork` and `exec`, redirection, pipes, lists, grouping and background execution. It does not have globbing, variables, quotes, a `PATH`, `&&` and `||`, job control, history and completion.

The program `args` showed that the expansion of `*`, of `$HOME` and the removal of the quotes are done by the shell, before the program starts: the kernel and the program only see the final list of arguments. Reading `sh.c` explained the observed behavior: `cd` is handled by the shell itself in the parent process (so it works only when it is the whole line), the programs are searched only in the current directory (so `ls` fails after `cd d` and `/ls` works), and no code for `*` or `$` exists. Therefore the comfort of `bash` is not a property of the operating system, but of the shell program.